1 Introduction

1.1 Algorithm

A well-defined computational procedure to solve a problem

- Problem specifies the desired input/output relationship

1.1.1 Example: Sorting problem

- Input: a sequence of n numbers
- Output: a permutation (reordering) of the input sequence

Instance: a particular input of the problem

1.2 Correctness

- An algorithm is said to be correct if, for every input instance, it halts with the correct output.
- Correct algorithm *solves* the given computational problem.

1.2.1 Example: Insertion sort

```
function INSERTION-SORT(A, n)
    for j ← 2 to n do
        key ← A[j]
        i ← j − 1
        while i > 0 and A[i] >key do
            A[i + 1] ← A[i]
            i ← i − 1
        A[i + 1] ←key
```

1.2.2 Loop invariants

Conditions and relationships that are satisfied by the variables and data structures at the end of each iteration of the loop

To use loop invariants to prove correctness, we must show:

- **Initialization**: it is true prior to the first iteration
- **Maintenance**: if it is true before an iteration, it remains true before the next iteration
- **Termination**: when the loop terminates, it gives us a useful property that helps show the correctness

1.2.3 Correctness of insertion sort

- Loop invariant: the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$ in sorted order
- Initialization: trivial (when $j = 2$, $A[1..j-1]$ has single element)
- Maintenance: need a loop invariant for the inner while loop
- Termination: ends when $j = n + 1$, $A[1..n]$ consists of the elements originally in $A[1..n]$ but in sorted order

1.3 Asymptotic analysis

- Ignore machine-dependant constants
- Consider the leading term of the formula

1.3.1 Types of analysis

- Worst-case: maximum time of algorithm on any input of size n
- Average-case: average time of algorithm.
  - Need assumption of statistical distribution of inputs
- Best-case

1.3.2 Θ-notation

Definition Θ-notation

$\Theta(g(n)) = f(n)$ if there exist positive constants c_1, c_2, n_0 such that

$$0 \le c_1 g(n) \le f(n) \le c_2 g(n)$$

for all $n \ge n_0$. We say $g(n)$ is an *asymptotic tight bound* for $f(n)$.

This drop low-order terms and ignore leading constants. In other words, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = c, c \in \mathbb{R}^+$ then $f \in \Theta(g)$.

1.3.3 Asymptotic analysis of insertion sort

- Worst: input reverse sorted $T(n) = \sum_{j=2}^{n} \Theta(j) = \Theta(n^2)$
- Average: permutations equally likely, $T(n) = \sum_{j=2}^{n} \Theta(j/2) = \Theta(n^2)$

1.3.4 Example: Fibonacci numbers

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{if } n \ge 2 \end{cases}$$

```
function FIB1(n)                          ▷ Naïve algorithm
    if n=0 then return 0
    if n=1 then return 1
    else return FIB1(n-1) + FIB1(n-2)
```

Let $T(n)$ denote *computer steps* needed to compute `fib1(n)`.

$$T(n) = T(n-1) + T(n-2) + 3 \text{ for } n > 1$$

Since $F_n = F_{n-1} + F_{n-2}$, $T(n) \ge F_n \approx 2^{0.694n}$. Therefore, $T(n)$ is *exponential*. Can we do better?

```
function FIB2(n)                          ▷ Memoization
    if n=0 then return 0
    Array f[0...n]
    f[0]=0, f[1]=1
    for i=2...n do
        f[i] = f[i-1] + f[i-2]
    return f[n]
```

This is $\Theta(n)$ algorithm.

1.3.5 Deeper analysis

- We assumed basic computer steps take a constant time
- Basic computer steps: branching, loading, storing, comparisons, simple arithmetic, etc

n-th Fibonacci number is not small. → It takes $\Theta(n)$ to add two n-bit numbers? So for large n, `fib2` is $\Theta(n^2)$ algorithm.

1.3.6 Why algorithm analysis?

- **Composability**: If an algorithm is proven correct and has a guarantee on its running time, we can reuse it as a subroutine
- **Scaling**: how the running time scales with the problem size
- **Algorithm design**: leads to insights for better algorithms
- **Understanding**
- **Complexity theory**: "How hard is problem X?"

constant	c	cubic	n^3
logarithmic	$\log(n)$	polynomial	cn^c
square root	$\sqrt{n}$	superpolynomial	$n^{\log(n)}$
linear	n	exponential	2^n
quasilinear	$n\log(n)$	doubly exponential	2^{2^n}
quadratic	n^2	tetration	$2^{2^{\cdot^{\cdot^{2^n}}}}$

Table 1. Asymptotic growth

Name	Relationship	Definition ($\forall n \geq n_0$)	$\lim_{n\to\infty} f(n)/g(n)$
Theta Θ	$g(n)$ is *asy.* tight bound of $f(n)$	$\exists c_1, c_2, n_0 \in \mathbb{R}^+$ s.t. $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$	$c \in \mathbb{R}^+$
Big Oh O	$g(n)$ is *asy.* upper bound of $f(n)$	$\exists c, n_0 \in \mathbb{R}^+$ s.t. $0 \leq f(n) \leq cg(n)$	$c \in \mathbb{R}^*$
Omega Ω	$g(n)$ is *asy.* lower bound of $f(n)$	$\exists c, n_0 \in \mathbb{R}^+$ s.t. $0 \leq cg(n) \leq f(n)$	$c > 0$ or ∞
Little Oh o	$f(n)$ is *asy.* smaller than $g(n)$	$\forall c \in \mathbb{R}^+ \exists n_0$ s.t. $0 \leq f(n) < cg(n)$	0
Little Omega ω	$f(n)$ is *asy.* larger than $g(n)$	$\forall c \in \mathbb{R}^+ \exists n_0$ s.t. $0 \leq cg(n) \leq f(n)$	∞

Table 2. Asymptotic notations.

1.3.7 Properties

Theorem

$$f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n)), f(n) = \Omega(g(n))$$

Let $f, g, h : \mathbb{N} \to \mathbb{R}^*$

- **Transitivity**: $f \in O(g), g \in O(h)$ then $f \in O(h)$
- **Duality**: $f \in O(g) \Leftrightarrow g \in \Omega(f)$, $f \in o(g) \Leftrightarrow g \in \omega(f)$
- $f \in \Theta(g)$ then $g \in \Theta(f)$
- Θ is equivalence relation
- $O(f + g) = O(\max\{f, g\})$

Theorem

$$\log n \in o(n^\alpha) \forall \alpha > 0 n^k \in o(2^n) \forall k > 0$$

2 Recurrences

2.1 Divide-and-Conquer design paradigm

1. **Divide** : divide a problem into subproblems
2. **Conquer**: solve the subproblems recursively
3. **Combine**: combine the subproblem solutions appropriately

2.1.1 Example: Merge sort

1. Divide: Divide the input array in 2 subarrays.
2. Conquer: Recursively sort 2 subarrays.
3. Combine: Merge 2 sorted subarrays.

```
function MERGE-SORT(A[1...n])
    if n>1 then return MERGE(MERGE-SORT(A[1...⌊n/2⌋]),
MERGE-SORT(A[⌊n/2⌋ + 1...n]))
    else return A
function MERGE(x[1...k], y[1...l])
    if k=0 then return y[1...l]
    if l=0 then return x[1...k]
    if x[1] ≤ y[1] then return x[1]∘ MERGE(x[2...k], y[1...l])
    else return y[1]∘ MERGE(x[1...k], y[2...l])
```

Analysis of Merge sort

$$T(n) = \underbrace{2}_{\text{\# subproblems}} T(\underbrace{n/2}_{\text{subproblem size}}) + \underbrace{\Theta(n)}_{\text{work dividing \& combining}}$$

2.1.2 Base case

- When the problem size gets down to a small constant, just use a brute force approach that takes constant time.
- $T(n) \leq c \ \forall n \leq n_0$

2.2 Solving recurrences

How do we analyze algorithms expressed in a recursive way?

2.2.1 Solve by unrolling

Unroll the recurrence until we reach base case.

$$\begin{aligned} T(n) &= cn + T(n-1) \\ &= cn + c(n-1) + \cdots + c \end{aligned}$$

Find the bounds: Top $n/2$ terms are each at least $cn/2$.

$$\begin{aligned} &(n/2)(cn/2) \leq T(n) \leq cn^2 \\ &\therefore T(n) = \Theta(n^2) \end{aligned}$$

2.2.2 Substitution method

- Guess the solution and use mathematical induction to prove it
- Show the upper and lower bounds separately
- Make sure to show *exact* same form of the inductive hypothesis
- Pick constants that satisfy initial conditions

$$T(n) = 4T(n/2) + n$$

Guess that $T(n) = O(n^2)$.

Trial 1. Assume that $T(k) \leq ck^2$ for $k < n$. Then

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &\leq 4c(n/2)^2 + n \\ &= cn^2 + n \end{aligned}$$

We should strengthen the inductive hypothesis by subtracting a low-order term.

Trial 2. Assume that $T(k) \leq c_1 k^2 - c_2 k$ for $k < n$. Then

$$\begin{aligned} T(n) &= 4T(n/2) + n \\ &= c_1 n^2 - 2c_2 n + n \\ &\leq \underbrace{c_1 n^2 - c_2 n}_{\text{exact same form!}} \text{ if } c_2 > 1 \end{aligned}$$

2.2.3 Recursion-tree method

It is **not** a rigorous proof, but can be used to **provide a good guess** for the substitution method.

1. Each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
2. Sum the costs within each level of the tree to obtain a set of per-level costs.
3. Sum all the per-level costs to determine the total cost of all levels of the recursion.

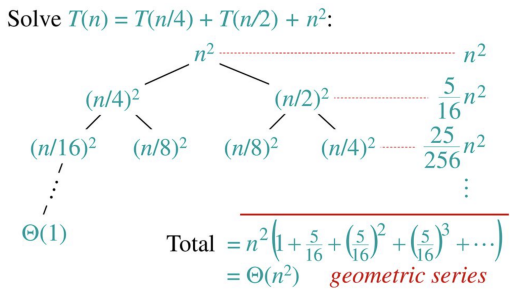


Figure 1. Recursion tree method

2.2.4 Master method

Consider divide-and-conquer style recurrence

$$T(n) = aT(n/b) + cn^k$$

First, we guess the running time using recursion tree:

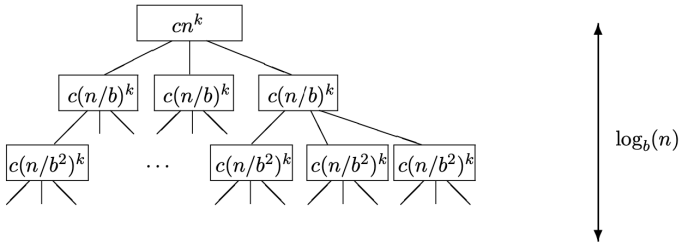


Figure 2. Recursion tree of general recurrence

Running time is

$$cn^k[1 + a/b^k + (a/b^k)^2 + \cdots_(a/b^k)^{\log_b n}]$$

By substituting $r = a/b^k$:

$$cn^k[1 + r + r^2 + \cdots r^{\log_b n}]$$

1. $r > 1$ then $T(n) \in \Theta(n^{\log_b a})$
2. $r = 1$ then $T(n) \in \Theta(n^k \log_b n)$
3. $r < 1$ then $T(n) \in \Theta(n^k)$

Generalized form of this is called Master theorem.

Theorem Master theorem

Let constants $a \geq 1, b > 1$, function $f(n) > 0$ and let $T(n) = aT(n/b) + f(n)$, then $T(n)$ is asymptotically bounded as follows:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, $T(n) = \Theta(n^{\log_b a})$
2. If $f(n) = \Theta(n^{\log_b a} \log n^k)$ for some constant $k > 0$, $T(n) = \Theta(n^{\log_b a} \log n^{k+1})$
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $f(n)$ satisfies the regularity condition *i.e.* $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, $T(n) = \Theta(f(n))$

Proof. We analyze each case separately:

Case 1: The cost is dominated by leaves, *i.e.* recursive calls, so $\Theta(n^{\log_b a})$.

Case 2: The cost is same across all levels: $n^{\log_b a} \log^{k+1} n$. The height of the recursion tree is $\log_b n$. Multiplying those two, we get $\Theta(n^{\log_b a} \log^{k+1} n)$.

Case 3: The regularity condition ensures that $f(n)$ provides an upper bound on the total cost. The cost is dominated by the root, *i.e.* the highest level, giving $\Theta(f(n))$. □

2.3 Example: Multiply two n-bit numbers

2.3.1 Naïve algorithms

- Adding a to itself b times: $0 \leq a, b \leq 2^n - 1$ so worst case 2^n additions, which takes $\Theta(n)$ so $\Theta(2^n \cdot n)$
- Adding itself for each 1: at most n times of addition, so $\Theta(n^2)$

Can we do better by divide-and-conquer?

$$X = 2^{n/2}A + B$$
$$Y = 2^{n/2}C + D$$

A	B
C	D

Figure 3. Dividing the problem

$$XY = 2^nAC + 2^{n/2}BC + 2^{n/2}AD + BD$$
$$T(n) = 4T(n/2) + cn$$

Using Master theorem, $\Theta(n^{\log_b a}) = \Theta(n^2)$. So this scheme doesn't improve running time.

2.3.2 Karatsuba algorithm

$$\begin{aligned} XY &= 2^nAC + 2^{n/2}BC + 2^{n/2}AD + BD \\ &= (2^n - 2^{n/2})AC + 2^{n/2}(A+B)(C+D) + (1 - 2^{n/2})BD \end{aligned}$$
$$T(n) = 3T(n/2) + c'n$$

Using Master theorem, $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

2.4 Example: Matrix multiplication

Input: $A = [a_{ij}], B = [b_{ij}].$
Output: $C = [c_{ij}] = A \cdot B.$
$i, j = 1, 2, \ldots, n.$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}$$

Figure 4. Problem definition

2.4.1 Standard algorithm

---

for $i \leftarrow 1$ to n **do**
  for $j \leftarrow 1$ to n **do**
    $c_{ij} \leftarrow 0$
    for $k \leftarrow 1$ to n **do**
      $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$

This is $\Theta(n^3)$ algorithm.

2.4.2 Divide-and-Conquer algorithm

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$\left.\begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dg \\ u &= cf + dh \end{aligned}\right\} \begin{aligned} &8 \text{ mults of } (n/2)\times(n/2) \text{ submatrices} \\ &4 \text{ adds of } (n/2)\times(n/2) \text{ submatrices} \end{aligned}$$

Figure 5. Divide-and-Conquer algorithm

$$T(n) = \underbrace{8}_{\#\text{ submatrices}} T(\underbrace{n/2}_{\text{submatrix size}}) + \underbrace{\Theta(n^2)}_{\text{Add submatrices}}$$

Using Master theorem, $\Theta(n^{\log_2 8}) = \Theta(n^3)$. So this scheme doesn't improve running time.

2.4.3 Strassen's algorithm

Multiply $2x2$ matrices with only 7 recursive calls.

$$T(n) = 7T(n/2) + \Theta(n^2)$$

2.5 Examples: $\Theta(\log n)$ algorithms

2.5.1 Binary search

Find an element in a sorted array:

1. Divide: Check middle element
2. Conquer: Recursively search subarray
3. Combine: Trivial

$$T(n) = T(n/2) + \Theta(1)$$

2.5.2 Powering a number

Compute a^n, where $n \in \mathbb{N}$

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if even} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if odd} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1)$$

2.5.3 Example: Fibonacci number

Theorem Recursive squaring

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

Proof. $n = 1$ is true by the definition. For $n \geq 2$,

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

proven by mathematical induction. □

The Theorem transforms problem into powering a number.

2.6 Quicksort

- Divide: Partition the array into two subarrays around a pivot x s.t. elements in lower subarray $\leq x \leq$ elements in upper subarray
- Conquer: Recursively sort the two subarrays
- Combine: Trivial

```
function QUICKSORT(A, p, r)                    ▷ Sorting subarray A[p...r]
    if p < r then
        q ←PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
function PARTITION(A, p, q)                    ▷ Θ(n)
    x ← A[p]
    i ← p
    for j ← p + 1 to q do
        if A[j] ≤ x then
            i ← i + 1
            Exchange A[i] ⇔ A[j]
    Exchange A[p] ⇔ A[i]
    return i
```

2.6.1 Analysis

Worst case is when one subarray has no element.

$$\begin{aligned} T(n) &= T(0) + T(n-1) + \Theta(n) \\ &= T(n-1) + \Theta(n) \\ &= \Theta(n^2) \end{aligned}$$

Best case is when PARTITION splits the array evenly.

$$\begin{aligned} T(n) &= 2(Tn/2) + \Theta(n) \\ &= \Theta(n \log n) \end{aligned}$$

What if the split is always $1 : 9$?

$$T(n) = T(\frac{1}{10}n) + T(\frac{9}{10}n) + \Theta(n)$$

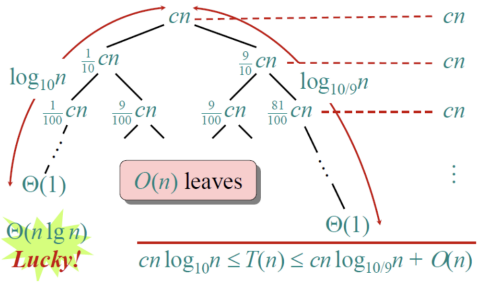


Figure 6. Recursion tree of quick sort

$$cn \log_{10} n \geq T(n) \geq cn \log_{10/9} n + O(n)$$

Also $\Theta(n \log n)$ algorithm.

2.6.2 Randomized quicksort

Partition around a random element.

```
function RAND-PARTITION(A, l, h)
    p ← random value ∈ [l, h]
    i ← p
    swap A[p] ⇔ A[h]
    for j ← l to h − 1 do ···                  ▷ Same
    Exchange A[i] ⇔ A[h]
    return i
```

- Running time is independent of the input order.
- The worst case is determined only by the output of a random-number generator.

Expected time analysis. Let $T(n) =$ *the random variable* for the running time of randomized quicksort on an input of size n, assuming random numbers are independent. For $k = 0, 1, \cdots, n-1$ the *indicator random variable* is defined as:

$$X_k = \begin{cases} 1 & \text{if RAND-PARTITION generates a } k : n-k-1 \text{ split,} \\ 0 & \text{otherwise} \end{cases}$$

$E[X_k] = Pr\{X_k = 1\} = 1/n$ since all splits are equally likely, assuming elements are distinct.

$$\begin{aligned} T(n) &= \begin{cases} T(0) + T(n-1) + \Theta(n) & \text{if } 0 : n-1 \text{ split,} \\ T(1) + T(n-2) + \Theta(n) & \text{if } 1 : n-2 \text{ split,} \\ \vdots \\ T(n-1) + T(0) + \Theta(n) & \text{if } n-1 : 0 \text{ split} \end{cases} \\ &= \sum_{k=0}^{n-1} X_k (T(k) + T(n-k-1) + \Theta(n)) \end{aligned}$$

By linearity of expectation,

$$\begin{aligned} E[T(n)] &= \sum_{k=0}^{n-1} E[X_k] \cdot E[T(k) + T(n-k-1) + \Theta(n)] \\ &= \frac{1}{n}\sum_{k=0}^{n-1} E[T(k)] + \frac{1}{n}\sum_{k=0}^{n-1} E[T(n-k-1)] + \frac{1}{n}\sum_{k=0}^{n-1} \Theta(n) \\ &= \frac{2}{n}\sum_{k=0}^{n-1} E[T(k)] + \Theta(n) \end{aligned}$$

$\Theta(n)$ absorbs $k = 0, 1$ terms, which gives:

$$E[T(n)] = \frac{2}{n}\sum_{k=2}^{n-1} E[T(k)] + \Theta(n)$$

Proof. We will use the fact that:

$$\sum_{k=2}^{n-1} k \log k \le \frac{1}{2}n^2 \log n - \frac{1}{8}n^2$$

Assume that $\forall k < n$, $E[T(k)] \le ak \log k$ for some $a > 0$. Then

$$\begin{aligned} E[T(n)] &\le \frac{2}{n}\sum_{k=2}^{n-1} ak \log k + \Theta(n) \\ &\le \frac{2a}{n}(\frac{1}{2}n^2 \log n - \frac{1}{8}n^2) + \Theta(n) \\ &= an \log n - (\frac{an}{4} - \Theta(n)) \\ &\le an \log n \end{aligned}$$

if a is chosen large enough so that $an/4$ dominates the $\Theta(n)$. □

3 Lower bound

3.1 Complexity of problems

Let $T_A(X)$ denote the running time of algorithm A given input X. The worst-case running time of A is

$$T_A(n) := \max_{|X|=n} T_A(X)$$

The worst-case complexity of a problem Π is the worst-case running time of the *fastest* algorithm for solving it:

$$T_\Pi(n) := \min_{A \text{ solves } \Pi} T_A(n) = \min_{A \text{ solves } \Pi} \max_{|X|=n} T_A(X)$$

Any algorithm A that solves Π implies an upper bound on the complexity of Π: $T_\Pi(n) \le T_A(n)$. Faster algorithms gives us better (smaller) upper bounds.

Then how can we argue that certain problems are hard (by proving lower bounds on their complexity)? To prove $T_\Pi(n) = \Omega(f(n))$, we must prove that every algorithm that solves Π has a worst-case running time $\Omega(f(n))$. So, we need to specify precisely what kinds of algorithms we will consider: called a *model of computation*.

3.2 Decision tree model

- Internal node: labeled by a query – a question about the input
- Edges: possible answers to the query
- Leaf: output

We want to define the running time of a k-ary decision tree algorithm. We can observe that the answers to the queries must give you enough information to specify any possible output.

If the problem has N different outputs, then the decision tree must have at least N leaves. Thus, if every query has at most k possible answers, the depth of the decision tree must be at least $\lceil \log_k N \rceil = \Omega(\log N)$.

3.2.1 Dictionary problem

Given an array A with n numbers and a number x, determine the position of x, if any.

Dictionary problem has $n+1$ possible outputs → at least $n+1$ leaves → depth at least $\lceil \log_k(n+1) \rceil = \Omega(\log N)$. So the complexity in the decision-tree model of computation is $\Omega(\log N)$ which matches the upper bound $O(\log N)$ from a perfectly-balanced binary search tree. Thus, binary search algorithm is optimal.

3.3 Lower bounds of sorting

Trivial $\Omega(n)$ to examine all the input.

We stick with *comparison sorts* - the sorted order are only determined by comparisons between the input elements.

3.3.1 Decision tree model of comparison sort

We only count comparison.

- Internal nodes $i : j$ represent comparison $a_i \le a_j$
  - Left subtree gives subsequent comparisons if $a_i \le a_j$
  - Right subtree gives subsequent comparisons if $a_i > a_j$
- Leaf: permutation $< \pi(1), \cdots, \pi(n) >$

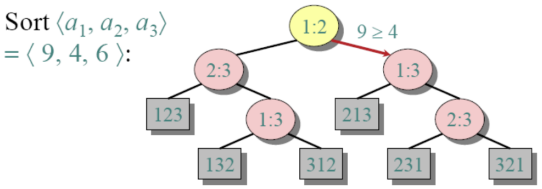


Figure 7. Example decision tree model of comparison sort

A decision tree can model the execution of any comparison sort.

- The running time of the algorithm = the length of the path taken
- Worst-case running time = height of tree

Theorem Lower bound of sorting

Any comparison sort algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof. There are $l \ge n!$ leaves on the decision tree, because every permutation appears at least once. Any binary tree of height h has $\le 2^h$ leaves.

$$\begin{aligned} n! \le l &\le 2^h \\ h &\ge \log(n!) \end{aligned}$$

From $\log(n!) = \Theta(n \log n)$ (Stirling's approximation), $h = \Omega(n \log n)$ □

Corollary

Heapsort, merge sort, and randomized quicksort are asymptotically optimal comparison sorts.

> Upper bound is algorithm-dependent, whereas lower bound is inherent to a problem.

3.4 Selection

- Input: A list of numbers S; an integer k
- Output: The kth smallest element of S

Naïve algorithm: Sort S, which is $O(n \log n)$.

3.4.1 Randomized divide-and-conquer

function RAND-SELECT(A, p, q, i) ▷ ith smallest of $A[p...q]$
  if $p = q$ **then return** $A[p]$
  $r \leftarrow$ RAND-PARTITION(A, p, q)
  $k \leftarrow r - p + 1$ ▷ k = rank($A[r]$)
  if $i = k$ **then return** $A[r]$
  if $i < k$ **then return** RAND-SELECT($A, p, r-1, i$)
  else return RAND-SELECT($A, r+1, q, i-k$)

Worst case is when pivot is chosen in a way that one subarray has no element.

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$$

Best case is when RAND-PARTITION splits the array evenly.

$$T(n) = T(n/2) + O(n) = O(n)$$

What if the split is always $1:9$?

$$T(n) = T(9n/10) + O(n) = O(n)$$

Average-case analysis Let's say that a pivot v is *good* if it lies within the 25th to 75th percentile of the array. It reduces the size of the subarray to at most $3/4$ of the size of the array. A randomly chosen pivot has a 50% chance of being good. After two split operations on average, the array will shrink to at most $3/4$ of its size. Let $T(n)$ be the expected running time on an array of size n,

$$T(n) \le T(3n/4) + \underbrace{T(n)}_{\text{time to reduce size to } \le 3n/4} = O(n)$$

Expected time analysis. Let $T(n)$ = the random variable for the running time of RAND-SELECT on an input of size n, assuming random numbers are independent. For $k = 0, 1, \cdots, n-1$, we define the indicator random variable X_k same as Section 2.6.2.

To obtain an upper bound, assume that the ith element always falls in the larger side of the partition:

$$T(n) = \begin{cases} T(\max(0, n-1)) + \Theta(n) & \text{if } 0 : n-1 \text{ split}, \\ T(\max(1, n-2)) + \Theta(n) & \text{if } 1 : n-2 \text{ split}, \\ \vdots \\ T(\max(n-1, 0) + \Theta(n) & \text{if } n-1 : 0 \text{ split} \end{cases}$$
$$= \sum_{k=0}^{n-1} X_k(\max(k, n-k-1)) + \Theta(n))$$

By linearity of expectation,

$$E[T(n)] = \sum_{k=0}^{n-1} E[X_k] \cdot E[T(\max(k, n-k-1)) + \Theta(n)]$$
$$= \frac{1}{n} \sum_{k=0}^{n-1} E[T(\max(k, n-k-1))] + \sum_{k=0}^{n-1} \Theta(n)$$
$$= \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + \Theta(n)$$

Proof. We will use the fact that:

$$\sum_{k=\lfloor n/2 \rfloor}^{n-1} k \le \frac{3}{8} n^2$$

Assume that $\forall k < n$, $E[T(k)] \le ck$ for some $c > 0$. Then

$$E[T(n)] \le \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + \Theta(n)$$
$$\le \frac{2c}{n} (\frac{3}{8} n^2) + \Theta(n)$$
$$= cn - \left(\frac{cn}{4} - \Theta(n) \right)$$
$$\le cn$$

if c is chosen large enough so that $cn/4$ dominates the $\Theta(n)$. □

3.4.2 Worst-case linear time algorithm

function SELECT($A[1...n], k$)
  if $n \le 25$ **then** use brute force
  else
    $m \leftarrow \lceil n/5 \rceil$ ▷ Divide to group of 5 elements
    for $i \leftarrow 1$ **to** m **do** ▷ For each group
      $B[i] \leftarrow$ SELECT($A[5i-4...5i], 3$) ▷ Median (brute force)
    $mom \leftarrow$SELECT($B[1...m], \lfloor m/2 \rfloor$) ▷ Recursion
    $r \leftarrow$PARTITION-AROUND($A[1...n], mom$)
    if $k < r$ **then return** SELECT($A[1...r-1], k$)
    else if $k > r$ **then return** SELECT($A[r+1...n], k-r$)
    else return mom

At least half the group medians are $\le mom$, which means at least $3n/10\rfloor$ elements are $\le mom$. Similarly, at least $3n/10\rfloor$ elements are $\ge mom$. Using SELECT, we recursively search at most $7n/10$ elements array in worst case.

$$T(n) \le \underbrace{T(n/5)}_{\text{SELECT}(B[1...m], \lfloor m/2 \rfloor)} + T(7n/10) + O(n) = O(n)$$